The Journal of Open Source Software

Zarr-Cesium: Client-side interactive 2D and 3D visualization of large multidimensional Zarr datasets in the browser

Tobias Ferreira [1]

1 National Oceanography Centre, UK

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Environmental sciences increasingly rely on multidimensional datasets describing oceanic, atmospheric, and terrestrial systems across space, depth, and time. Numerical models and remote sensing products routinely produce terabyte-scale data cubes with dimensions such as longitude, latitude, elevation, and time (Eyring et al., 2016; Hersbach et al., 2020). These datasets are commonly stored in chunked, cloud-optimized formats such as Zarr (Zarr Developers, 2026b), enabling scalable storage and parallel analysis.

Although storage and computational frameworks have evolved to support these multidimensional data structures (Abernathey et al., 2021; Zarr Developers, 2026b), visualization workflows often transform them into two-dimensional raster products for web delivery, separating rendered outputs from the underlying data cube.

`Zarr-Cesium` is a TypeScript library that addresses these limitations. It enables interactive, browser-native visualization of multidimensional Zarr datasets within a 3D geospatial environment using CesiumJS (Cesium GS, 2026). The software streams chunked array data directly from object storage into the browser and renders it using WebGL2, without requiring backend services, tile-generation pipelines, or intermediate raster products.

The toolkit supports 2D scalar fields, 3D volumetric slicing, and animated vector fields. It is compatible with Zarr v2 and v3, multiscale pyramids (Zarr Developers, 2026a; **?**), and CF-compliant metadata conventions (Eaton et al., 2025). By integrating cloud-native data access with GPU-accelerated rendering, `Zarr-Cesium` enables interactive exploration of large scientific datasets directly from their cloud-hosted storage.

Statement of Need

Although storage and computation frameworks have matured, visualization workflows remain fragmented across several partially overlapping ecosystems.

Scientific communication frequently relies on static figures or screenshots, which flatten inherently multidimensional systems into fixed two-dimensional views. Server-side tile-generation platforms convert datasets into raster products through Web Map Service (WMS) (**?**) or custom APIs, requiring persistent backend infrastructure, preprocessing steps, and operational maintenance. These workflows also duplicate data into derived tile products, increasing storage requirements and separating rendered output from its canonical source.

Three-dimensional visualization is frequently essential for scientific interpretation. Many environmental processes exhibit strong vertical structure, including ocean stratification and thermocline variability, mixed-layer dynamics, and large-scale overturning circulation (**?**). Atmospheric systems similarly depend on vertical structure, including boundary layer evolution

DRAFT

The Journal of Open Source Software

and convection (**?**). Horizontal maps alone can obscure depth-dependent gradients, vertical shear, and subsurface anomalies that are central to physical interpretation. Interpretation of velocity fields further requires contextualizing transport pathways and circulation patterns across depth and time (**?**).

As multidimensional datasets become standard in operational modeling and reanalysis systems, there is a need for visualization approaches that preserve their native structure rather than flattening or duplicating them into derived products.

Statement of the Field

Existing visualization ecosystems address parts of this problem but do not provide a unified client-side solution for multidimensional Zarr datasets.

Server-side tile-generation platforms convert multidimensional datasets into raster products exposed through WMS endpoints or custom APIs. Systems such as GeoServer (GeoServer, 2026) and ncWMS (Blower et al., 2013) enable serving NetCDF-based datasets as map layers but require persistent backend infrastructure and preprocessing steps to generate tiles.

Cloud-native raster ecosystems based on Cloud-Optimized GeoTIFFs (COGs) (**?**) enable efficient streaming of large two-dimensional imagery products. Tools such as TiTiler (Development Seed, 2026a) provide scalable server-side generation of map tiles directly from COGs. However, these approaches are fundamentally oriented toward two-dimensional raster data. Extending them to higher-dimensional data cubes typically requires precomputed slicing, separate endpoints for each non-spatial dimension, or additional backend logic to manage depth and time selection.

For Zarr datasets, backend tile-generation solutions such as titiler-xarray (Development Seed, 2026b) and xpublish-tiles (Earthmover, 2026) expose Zarr-backed arrays through tile APIs. While these tools allow dynamic slicing of multidimensional data, they still depend on server-side infrastructure to perform array access, rasterization, and tile generation. Visualization therefore remains mediated by backend services, and rendered products are transmitted as image tiles rather than structured array subsets.

Client-side geospatial engines such as CesiumJS (Cesium GS, 2026) provide powerful 3D globe visualization but do not natively support streaming and slicing of multidimensional chunked array formats such as Zarr. Conversely, browser-based Zarr libraries such as Zarrita (Zarrita Developers, 2026) enable direct access to chunked arrays in JavaScript but do not provide geospatial integration, volumetric slicing, or GPU-based colormapping pipelines.

Consequently, no general-purpose framework currently supports direct streaming of multidimensional Zarr arrays from object storage, automatic interpretation of CF-compliant metadata, dynamic slice computation in the browser, GPU-based rendering of scalar and vector fields, and fully static deployment without backend services. `Zarr-Cesium` fills this gap by integrating these capabilities into a coherent client-side architecture.

Software Design

`Zarr-Cesium` is implemented in TypeScript and organized around three primary provider classes: `ZarrLayerProvider` for 2D scalar fields, `ZarrCubeProvider` for 3D volumetric slicing, and `ZarrCubeVelocityProvider` for animated vector fields. These providers encapsulate data access, multidimensional slicing logic, and rendering integration.

All providers access Zarr datasets via a browser-based `FetchStore`, supporting HTTP range requests, consolidated metadata, and both Zarr v2 and v3 specifications. Multiscale pyramid datasets are detected through `multiscales` metadata, and resolution levels are selected dynamically based on view scale. A lightweight in-memory cache maintains recently accessed multiscale levels.

DRAFT

Dimension identification is performed automatically using CF `standard_name` attributes (Eaton et al., 2025) and configurable alias mappings. Geographic bounds from Cesium tiles or camera extents are translated into array index ranges, with automatic CRS detection between EPSG:4326 and EPSG:3857. Zarr slice arguments are constructed dynamically, and value-based selectors (e.g., timestamps or depths) are converted to nearest-index selections.

To maintain interactivity, the system limits concurrent chunk requests and uses `AbortController` instances to cancel stale requests during rapid navigation. Only visible spatial subsets are requested, and multiscale level selection reduces resolution at broader zoom levels.

Rendering is performed using WebGL2. For scalar fields, float32 array subsets are uploaded as `R32F` textures and processed in fragment shaders that apply scale-factor correction, normalization, no-data masking, and dynamic colormapping. Because scaling and colormapping occur on the GPU, visual parameters can be adjusted without re-fetching data.

For volumetric cubes, array subsets are reshaped into ndarray structures and horizontal or vertical slices are generated dynamically. Textured Cesium `Primitive` geometries are constructed, and elevation values are converted into globe-compatible heights with optional vertical exaggeration or subsurface offsets. Slices are regenerated when dimension selectors or styling parameters change.

Velocity fields are handled by loading U and V components from separate Zarr stores, extracting elevation slices, and passing the data to the `cesium-wind-layer` library (Cesium Wind Layer Developers, 2024) for particle-based animation. Multiple elevation layers can be rendered simultaneously, enabling stacked three-dimensional flow visualizations.

By combining selective chunk loading, multiscale resolution management, GPU-based rendering, and minimal data duplication, the architecture enables interactive exploration of datasets significantly larger than browser memory limits. Performance is constrained primarily by chunk size and network bandwidth rather than total dataset size.

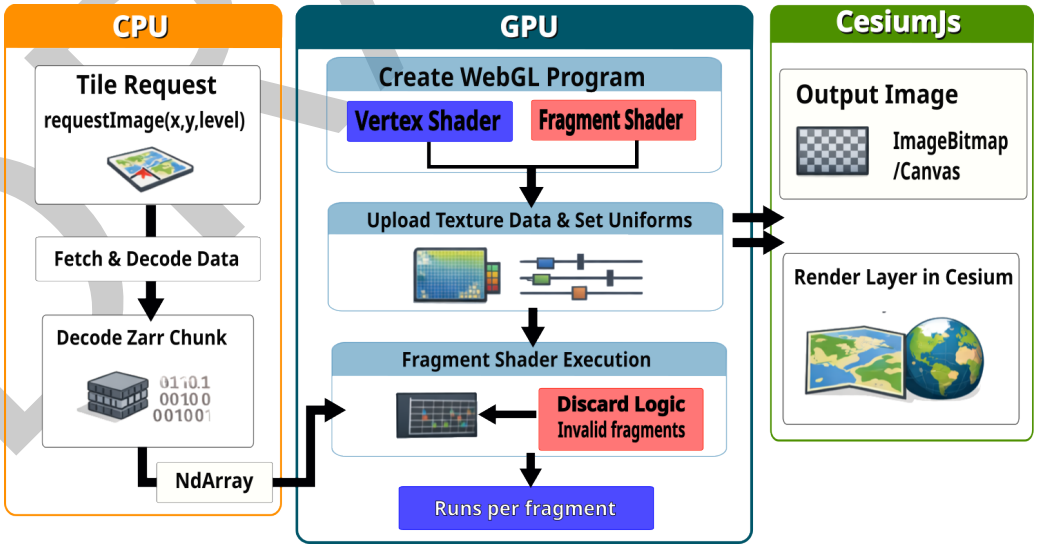


Figure 1: Representation of the workflow of the Zarr-Cesium package, from tile request to WebGL layer generation on the map.

Research Impact Statement

`Zarr-Cesium` integrates cloud-native chunked storage access, CF-aware dimension resolution, multiscale pyramid handling, and GPU-based rendering into a unified client-side architecture.

DRAFT

The software was developed to meet an operational need for interactive visualization of multidimensional Zarr datasets stored in object storage without server-side infrastructure. It serves as the core rendering engine of the AtlantiS visualisation tool at the National Oceanography Centre, enabling browser-based exploration of multidimensional oceanographic datasets in a 3D geospatial environment through fully client-side streaming and rendering. It has been released as an open-source TypeScript library to enable broader adoption across the environmental sciences and related domains.

`Zarr-Cesium` fills a clear gap in the ecosystem. No existing framework integrates client-side multidimensional slicing, CF-aware metadata interpretation, WebGL2-based GPU rendering, and CesiumJS 3D geospatial visualization into a unified, backend-free system. Beyond reducing infrastructure complexity, this architecture supports more transparent and exploratory analysis of inherently multidimensional environmental systems. Interactive slicing and dynamic rendering allow datasets to be examined in their full spatial and temporal structure rather than as predefined two-dimensional views. As cloud-native storage formats such as Zarr v3 and GeoZarr gain adoption, coupling scalable object storage with browser-native visualization becomes an essential component of modern scientific data dissemination. `Zarr-Cesium` provides such a component, lowering barriers to interactive publication and enabling reproducible, infrastructure-light access to large environmental datasets.

AI usage disclosure

Portions of this manuscript were edited with the assistance of a large language model for clarity and structure. All technical content, software design decisions, and scientific claims were written and verified by the author. No AI-generated code was included in the software without manual review and testing.

Acknowledgements

This work builds upon the open-source efforts of the CesiumJS, Zarrita, and cesium-wind-layer communities. Storage and testing infrastructure were supported by JASMIN, the UK's environmental data analysis facility. The project forms part of the AtlantiS initiative led by the National Oceanography Centre.

References

Abernathey, R., Augspurger, T., Banihirwe, A., Blackmon-Luca, C. C., Crone, T., Gentemann, C., Hamman, J., Henderson, N., Lepore, C., McCaie, T., Robinson, N., & Signell, R. P. (2021). Cloud-native repositories for big scientific data. *Computing in Science & Engineering*, *23*(2), 26–35. https://doi.org/10.1109/MCSE.2021.3059437

Blower, J. D., Gemmell, A. L., Griffiths, G. H., Haines, K., Santokhee, A., & Yang, X. (2013). A web map service implementation for the visualization of multidimensional gridded environmental data. *Environmental Modelling & Software*, *47*, 218–224. https://doi.org/10.1016/j.envsoft.2013.04.002

Cesium GS. (2026). *CesiumJS*. https://cesium.com/platform/cesiumjs/

Cesium Wind Layer Developers. (2024). *cesium-wind-layer*. https://github.com/hongfaqiu/cesium-wind-layer

Development Seed. (2026a). *TiTiler: A modern dynamic tile server for raster data*. https://github.com/developmentseed/titiler

Development Seed. (2026b). *titiler-xarray*. https://github.com/developmentseed/titiler-xarray

DRAFT

Earthmover. (2026). *xpublish-tiles*. https://github.com/earth-mover/xpublish-tiles

Eaton, B., Gregory, J., Drach, B., Taylor, K., Hankin, S., Caron, J., Signell, R., Bentley, P., Rappa, G., Höck, H., Pamment, A., Juckes, M., Raspaud, M., Blower, J., Horne, R., Whiteaker, T., Blodgett, D., Zender, C., Lee, D., … Laake, P. V. (2025). *NetCDF Climate and Forecast (CF) Metadata Conventions* (Version 1.13). CF Community. https://doi.org/10.5281/zenodo.17801666

Eyring, V., Bony, S., Meehl, G. A., Senior, C. A., Stevens, B., Stouffer, R. J., & Taylor, K. E. (2016). Overview of the coupled model intercomparison project phase 6 (CMIP6) experimental design and organization. *Geoscientific Model Development*, *9*, 1937–1958. https://doi.org/10.5194/gmd-9-1937-2016

GeoServer. (2026). *GeoServer*. https://geoserver.org/

Hersbach, H., Bell, B., Berrisford, P., Hirahara, S., Horanyi, A., Munoz-Sabater, J., Nicolas, J., Peubey, C., Radu, R., Schepers, D., & others. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, *146*(730), 1999–2049. https://doi.org/10.1002/qj.3803

Zarr Developers. (2026a). *GeoZarr specification*. https://github.com/zarr-developers/geozarr-spec

Zarr Developers. (2026b). *Zarr: Open standard for chunked, compressed n-dimensional arrays*. https://zarr.dev/

Zarrita Developers. (2026). *Zarrita: A minimal and modular Zarr implementation in TypeScript*. https://github.com/manzt/zarrita.js

DRAFT